

CSE327

Software Engineering

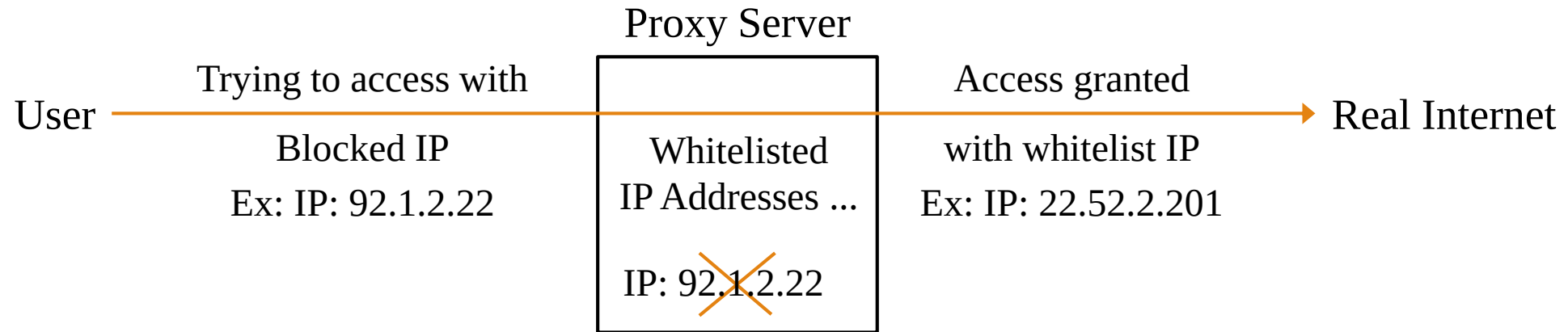
AKM IQTIDAR NEWAZ [IQN]

LECTURE - 13



Proxy Design Pattern

- ❑ Proxy is a **structural design pattern** that lets you provide a substitute or place holder for another object. A proxy controls access to the original object, allowing you to perform something either before or after the request gets through to the original object.



Key Components

□ Subject Interface:

- ❖ Defines the common interface for the proxy and real object.

□ Real Subject:

- ❖ The actual subject that performs the real operation.

□ Proxy:

- ❖ Acts as an intermediary.
- ❖ Adds functionality (e.g., access control) before delegating the request to the real subject.

□ Client:

- ❖ The client interacts with the proxy, which in turn interacts with the real object.

Example 1 (Internet Connection)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Abstract class or Interface
class Internet
{
public:
    virtual void connectTo(const string &serverHost) const = 0;
    virtual ~Internet() = default;
};

// Real Internet Class
class RealInternet : public Internet
{
public:
    void connectTo(const string &serverHost) const override
    {
        cout << "Connecting to " << serverHost << endl;
    }
};
```

Main.cpp

Example 1 (Internet Connection)

```
// Proxy Internet Class
class ProxyInternet : public Internet
{
private:
    RealInternet *realInternet;
    static vector<string> bannedSites;

public:
    ProxyInternet() : realInternet(new RealInternet()) {}
    void connectTo(const string &serverHost) const override
    {
        string lowerServerHost = serverHost;
        transform(lowerServerHost.begin(), lowerServerHost.end(), lowerServerHost.begin(), ::tolower);
        if (find(bannedSites.begin(), bannedSites.end(), lowerServerHost) != bannedSites.end())
        {
            throw runtime_error("Access Denied to "+ serverHost);
        }
    }
};

// Initializing static banned sites
vector<string> ProxyInternet::bannedSites = {"abc.com", "xyz.com", "banned.com"};
```

Main.cpp

Example 1 (Internet Connection)

```
// Main Function
template <typename T>
void attemptConnection(T &internet, const string &site)
{
    try
    {
        internet.connectTo(site);
        cout << "Successfully connected to " << site << "!" << endl;
    }
    catch (const exception &e)
    {
        cout << e.what() << endl;
    }
}

int main()
{
    ProxyInternet internet;
    attemptConnection(internet, "example.com");
    attemptConnection(internet, "abc.com"); // This site is banned.
    attemptConnection(internet, "stackoverflow.com");
}
```

Main.cpp

Output:

Successfully connected to example.com!

Access Denied to abc.com

Successfully connected to stackoverflow.com!

Example 2 (Image Downloader)

```
#include <iostream>
using namespace std;

// Abstract class or Interface
class Image
{
public:
    virtual void display() const = 0;
    virtual ~Image() = default;
};

// Actual Subject
class RealImage : public Image
{
private:
    string fileName;
    void load() const
    {
        cout << "Loading image: " << fileName << endl;
    }

public:
    RealImage(const string &filename) : fileName(filename)
    {
        load();
    }
    void display() const override
    {
        cout << "Displaying image: " << fileName << endl;
    }
};
```

Main.cpp

Example 2 (Image Downloader)

```
// Proxy Subject
class ProxyImage : public Image
{
public:
    string fileName;
    mutable RealImage *realImage;

public:
    ProxyImage(const string &filename) : fileName(filename), realImage(nullptr) {}
    ~ProxyImage()
    {
        delete realImage;
    }
    void display() const override
    {
        if (realImage == nullptr)
        {
            // Lazy initialization
            realImage = new RealImage(fileName);
        }
        realImage->display();
    }
};
```

Main.cpp

Example 2 (Image Downloader)

Main.cpp

```
int main()
{
    ProxyImage proxy("dogs.png");
    proxy.display();
}
```

Output:

Loading image: dogs.png

Displaying image: dogs.png